

Domain 5 Lesson 1

Python v2

Syntax Errors

Syntax errors in code are similar to punctuation and spelling errors in writing. In programming, syntax errors cause code not to run. Syntax errors prevent code from being compiled and executed. Python's compiler checks for syntax correctness before running any code.

Purpose

Upon completing this project, you will better understand how to recognize syntax errors.

Steps for Completion

1. Open the **511-syntax.py** file from your Domain 5 Student folder.
2. Run the code as many times as needed to help identify the syntax errors within the code.
3. Edit the code to remove the three syntax errors that are present.
4. Save the file as **511-syntax-completed**

Project Details

Project file

511-syntax.py

Estimated completion time

5 minutes

Video reference

Domain 5

Topic: Analyze, Detect, and Fix Errors

Subtopic: Syntax Errors

Objectives covered

5 Troubleshooting and Error Handling

5.1 Analyze, detect, and fix code segments that have errors

5.1.1 Syntax errors

Logic Errors

With a logic error, there is no syntax problem with the code, and the code does not cause a crash, but it does not perform the way one expects it to when the code is run. Logic errors can be difficult to identify because they do not return any error messages.

Purpose

Upon completing this project, you will better understand how to recognize logic errors.

Steps for Completion

1. Open the **512-logic.py** file from your Domain 5 Student folder.
2. Run the code. Does the result match the expected result in the comment on line 5?
 - a. It doesn't
3. Edit the code to remove the logic error that is present.
4. Save the file as **512-logic-completed**

Project Details

Project file

512-logic.py

Estimated completion time

5 minutes

Video reference

Domain 5

Topic: Analyze, Detect, and Fix Errors

Subtopic: Logic Errors

Objectives covered

5 Troubleshooting and Error Handling

5.1 Analyze, detect, and fix code segments that have errors

5.1.2 Logic errors

Runtime Errors

Runtime errors occur when one attempts to run code. They occur when one tries to do something with an application that cannot be done as the application's code is being run. Runtime errors can be a result of logic errors.

Purpose

Upon completing this project, you will better understand how to recognize runtime errors.

Steps for Completion

1. Open the **513-runtime.py** file from your Domain 5 Student folder.
2. Run the code to identify the runtime error present in the code.
3. Edit the code to remove the runtime error.
4. Save the file as **513-runtime-completed**

Project Details

Project file

513-runtime.py

Estimated completion time

5 minutes

Video reference

Domain 5

Topic: Analyze, Detect, and Fix Errors

Subtopic: Runtime Errors

Objectives covered

5 Troubleshooting and Error Handling

5.1 Analyze, detect, and fix code segments that have errors

5.1.3 Runtime errors

Review 5.1

Purpose

Upon completing this project, you will better understand how to identify, find, and fix errors in code.

Steps for Completion

1. Open the **513-analyze.py** file from your Domain 5 Student folder.
2. Run the code. Where is there an error, and what type of error is it?
 - a. Line 7, syntax error
3. What is the error?
 - a. There's a space between f and the quotation marks
4. Fix the error and rerun the code. Are there any other errors?
 - a. Broken operator in 8th line
5. Identify the new error.
 - a. Logic error
6. Fix the error and rerun the code. Are there any other errors?
 - a. No
7. Return to the videos to compare your answers with those in the videos.

Project Details

Project file

513-analyze.py

Estimated completion time

10 minutes

Video reference

Domain 5

Topic: Analyze, Detect, and Fix Errors

Subtopic: Runtime Errors; Review 5.1

Objectives covered

5 Troubleshooting and Error Handling

5.1 Analyze, detect, and fix code segments that have errors.

Domain 5 Lesson 2

Python v2

Exception Handling: try, except, else, and finally

Many programming languages, Python included, have keywords that can be used to attempt to run code, control the error message displayed if the code would otherwise cause a crash, and then run certain logic depending on whether the original intent to run code worked. Controlling errors and their messages is known as exception handling. Four common exception-handling keywords often used together are try, except, else, and finally.

Purpose

Upon completing this project, you will better understand how to use the try, except, else, and finally keywords to manage exception handling.

Steps for Completion

1. Review the code, then answer the questions regarding the try, except, else, and finally statements in the code.

```
524-finally.py > ...
1 figuratives = ['Simile', 'Metaphor', 'Personification', 'Hyperbole', 'Allusion']
2 try:
3     figurative_input = int(input('Enter a number from 1-5 to get an example '))
4     figurative = figuratives[figurative_input-1]
5 except:
6     print('You did not enter a figurative. Try again')
7 else:
8     print(f'You chose the {figurative} figurative and will get an example soon.')
9 finally:
10    print('Thank you for playing.')
```

- a. What is the purpose of the try keyword on line 2?

- i. To attempt to run code

- b. What is the purpose of the except keyword on line 5?

- i. It runs in case of exceptions in the code

- c. What is the purpose of the else keyword on line 7?

- i. It runs after the try code (if the try code is successful)

- d. What is the purpose of the finally keyword on line 9?

- i. It's the final ran thing after everything else.

Project Details

Project file

N/A

Estimated completion time

10 minutes

Video reference

Domain 5

Topic: Exception Handling

Subtopic: try; except; else in
Exception Handling; finally

Objectives covered

5 Troubleshooting and Error Handling

5.2 Analyze and construct code
segments that handle exceptions

5.2.1 try

5.2.2 except

5.2.3 else

5.2.4 finally

Exception Handling: raise

The raise keyword is another keyword commonly used for exception handling. Raising errors allows a developer to control the type of error to raise and log it if necessary.

Purpose

Upon completing this project, you will better understand how to use the raise keyword to manage exception handling.

Steps for Completion

1. Review the code, then answer the questions regarding the raise keyword in the code.

```
525-raise.py > temperature
1 temperature = float(input("What temperature is absolute zero "))
2 if temperature > 0:
3     raise ValueError("The temperature has to be negative")
4 else:
5     print("The temperature is negative, yes, -459.67 F to be exact.")
```

- a. What will happen if a negative number is entered for the temperature variable?
 - i. It will run the code inside else
- b. What will happen if a positive number is entered for the temperature variable?
 - i. It will throw an error

Project Details

Project file

N/A

Estimated completion time

5 minutes

Video reference

Domain 5

Topic: Exception Handling

Subtopic: raise

Objectives covered

5 Troubleshooting and Error Handling

5.2 Analyze and construct code segments that handle exceptions

5.2.5 raise

Review 5.2

Purpose

Upon completing this project, you will better understand the logic flow of exception handling, specifically try-except-else-finally blocks of code.

Steps for Completion

1. Open the **525-analyze.py** file from your Domain 5 Student folder.
2. Which blocks of code will run as the code is currently written?
 - a. Try, except and finally
3. Change the games variable to **2**
4. Which blocks of code will run after the change in the games variable?
 - a. Try, else, finally

Project Details

Project file

525-analyze.py

Estimated completion time

5 minutes

Video reference

Domain 5

Topic: Exception Handling

Subtopic: raise; Review 5.2

Objectives covered

5 Troubleshooting and Error Handling

5.2 Analyze and construct code segments that handle exceptions

Domain 5 Lesson 3

Python v2

Assert Methods

Unit testing is done on a small unit of code to check its functionality. One way of performing unit testing is to use assert methods. Common assert methods include assertEquals, assertTrue, assertIs, assertIn, and assertIsInstance.

Purpose

Upon completing this project, you will better understand assert methods.

Steps for Completion

1. Review the code, then answer the questions regarding the assert methods present in the code.

```
534-assert.py > TestMain > test_location
1  import unittest
2
3  class TestMain(unittest.TestCase):
4
5      def test_location(self):
6          a = 'red'
7          b = 'red'
8          self.assertEqual(a,b)
9
10     def test_truth(self):
11         self.assertTrue(2 + 5 * 3 == 21)
12
13     def test_is(self):
14         a = 'red'
```

- a. What action does the code on line 8 perform?

- i. It tests if the two parameters given are equal

- b. What action does the code on line 11 perform?

- i. Tests if the condition is true

```
13     def test_is(self):
14         a = 'red'
15         b = 'red'
16         self.assertIs(a,b)
17
18     def test_in(self):
19         a = 'red'
20         b = ['red', 'green', 'blue']
21         self.assertIn(a,b)
```

- c. What action does the code on line 16 perform?

- i. Tests if the two are the same

- d. What action does the code on line 21 perform?

- i. Tests if a is inside b

Project Details

Project file

N/A

Estimated completion time

10 minutes

Video reference

Domain 5

Topic: Perform Unit Testing

Subtopic: Unittest; Assert Methods

Objectives covered

5 Troubleshooting and Error Handling

5.3 Perform unit testing

5.3.1 Unittest

5.3.4 Assert methods

(assertIsInstance, assertEquals, assertTrue, assertIs, assertIn)

Functions and Methods

One of the most common types of unit tests is testing a function to see if it performs as it should. Because classes are often reused in code, it is important that a class's methods be tested for accuracy as well.

Purpose

Upon completing this project, you will better understand the importance of unit testing on functions and methods.

Steps for Completion

1. To test a function or method correctly, one must know what behavior are expected.
2. All code should be tested before an application is released to production.
3. Methods are functions that belong to classes.
4. Classes are frameworks for objects.

Project Details

Project file

N/A

Estimated completion time

5 minutes

Video reference

Domain 5

Topic: Perform Unit Testing

Subtopic: Functions; Methods

Objectives covered

5 Troubleshooting and Error Handling

5.3 Perform unit testing

5.3.2 Functions

5.3.3 Methods

Review 5.3

Purpose

Upon completing this project, you will better understand which assert test to use in a given situation and predict whether a test will pass or fail.

Steps for Completion

1. Open the **534-analyze.py** file from your Domain 5 Student folder.
2. Replace the comment on line 8 with the assert test needed to test whether variable a is contained within variable b.
3. Which test is needed for this testing situation?
 - a. assertIn
4. Will the test pass in this situation?
 - a. No
5. Return to the videos to compare your answers with those found in the videos.

Project Details

Project file

534-analyze.py

Estimated completion time

5 minutes

Video reference

Domain 5

Topic: Perform Unit Testing

Subtopic: Assert Methods; Review 5.3

Objectives covered

5 Troubleshooting and Error Handling

5.3 Perform unit testing